Who Does the Work? Dissecting Physics-Inspired Graph Neural Networks for Graph Colouring

Semyon Guretskiy
336249255

Ori Petel
325211498

Adi Shimoni
316342435

Abstract

Physics-inspired graph neural networks (PI-GNNs) colour a graph by training a fresh network to minimise a Potts relaxation of the conflict count, then rounding its soft output. Credit for a good colouring could go to three bundled ingredients: the neural optimiser, the information in its soft output, or the decoding and repair applied afterwards. We separate them with controlled interventions on 68 DIMACS graphs with one colour fewer than the chromatic number. We find that (i) training approaches discrete local minima but rarely reaches them: only 42% of runs are single-flip optimal after 10^5 steps, and wrong vertices are as confident as right ones, which an exact softmax-gradient identity explains; (ii) the network is not what makes PI-GNN work: optimising one logit vector per vertex gives better raw colourings, PI-GNN's modest advantage after repair vanishes without dropout, and directly noise-injected logits beat PI-GNN on 44 of 68 graphs; (iii) the learned probabilities carry real coordinated information, yet a graph-only decoder given no learned information does as well overall. The optimiser, not the architecture or the decoder, decides the outcome.

1 Introduction

A k-colouring of $G = (V, E)$ gives every vertex one of k colours; an edge whose endpoints share a colour is a *conflict*, and the chromatic number $\chi(G)$ is the fewest colours allowing none. Minimising conflicts is NP-hard, and unsupervised graph neural networks are a recent line of attack. The physics-inspired GNN (PI-GNN) of Schuetz et al. (2022b) trains, for each graph, a new network whose softmax output is a *soft colouring* P, minimises a Potts energy equal to the expected number of conflicts, and rounds P to a hard colouring.

When such a pipeline produces a good colouring, *what deserves the credit*? Figure 1 shows

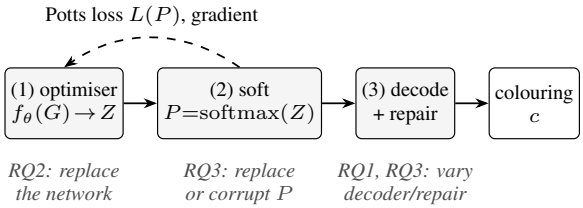


Figure 1: The three ingredients of PI-GNN colouring and where each research question intervenes. The graph enters both the network and the loss.

three ingredients that are always evaluated together: (1) the neural parameterisation and its optimiser, (2) the information in the soft output P, and (3) the decoder and any local repair. Critiques showing that greedy algorithms match or beat PI-GNNs (Angelini and Ricci-Tersenghi, 2023; Boettcher, 2023) tell us *that* the pipeline can fall short, not *which* part is responsible. We ask:

> *Which ingredient of PI-GNN is responsible for the quality of its colourings: the GNN optimiser, the learned soft output, or the decoding and repair?*

We split it into three sub-questions, each answered by intervening on one ingredient while holding the others fixed (Figure 1). **RQ1** asks what training actually produces: does the decoded output approach a discrete local minimum, and how good is that minimum? **RQ2** asks whether the GNN is needed: does it beat optimising one free logit vector per vertex on the same loss? **RQ3** asks whether the learned P helps a stronger decoder more than uninformative or corrupted inputs do.

All experiments use $k = \chi(G) - 1$ colours. Zero conflicts are then impossible, so methods are compared on a graded conflict count rather than a binary success rate that would saturate on easy graphs. Our contribution is a controlled decomposition protocol (identical starting outputs, sepa-

rately tuned competitors, matched steps *and* time, one shared repair, graph-level paired statistics) and the answer it gives: training makes outputs confident long before it makes them locally optimal; the network's edge over free logits is small, depends on dropout, and is overturned by plain noise; and the learned P beats corrupted inputs but not a graph-only decoder.

2 Background and Related Work

Learning for combinatorial optimisation. Learned methods either construct solutions or guide classical solvers (Bengio et al., 2021); for colouring, Lemos et al. (2019) trained a supervised GNN to decide k-colourability. Unsupervised approaches instead minimise a differentiable relaxation per instance. Karalias and Loukas (2020) relax the objective to an expectation under independent vertex distributions and recover a solution by derandomised rounding (Raghavan, 1988), with a guarantee in terms of the relaxed loss. PI-GNN (Schuetz et al., 2022a,b) applies the same idea with a QUBO or Potts energy (Wu, 1982) and a GraphSAGE network (Hamilton et al., 2017), reporting results competitive with classical solvers on DIMACS benchmarks (Johnson and Trick, 1996).

Critiques and ablations. Angelini and Ricci-Tersenghi (2023) and Boettcher (2023) showed that simple greedy algorithms match or beat PI-GNN on maximum independent set and max-cut. These are end-to-end comparisons: they show the pipeline can lose, not which component fails or why. Vanderbush and Weber (2026) include a direct-logit (depth-zero) ablation of a modified PI-GNN for colouring. We make that comparison the central object and control what an ablation alone leaves open: a shared starting point, separate tuning, matched time, noise controls and a common repair step. To our knowledge, no study has isolated the contribution of each PI-GNN ingredient.

Classical references and training dynamics. DSATUR (Brélaz, 1979) greedily colours the most constrained vertex first. TabuCol (Hertz and de Werra, 1987) is a local search that accepts worsening moves and forbids recent reversals, and underlies state-of-the-art hybrids (Galinier and Hao, 1999); we use both as reference points. A network moves its output through shared parameters, so one step moves many vertices at once (Jacot et al., 2018), and dropout (Srivastava et al., 2014) injects noise into training; both matter for RQ2.

3 Problem, Tools and Protocol

Relaxation. A soft colouring $P \in \mathbb{R}^{n\times k}$ has a probability row p_v per vertex, obtained from logits by $P = \mathrm{softmax}(Z)$. The Potts loss and hard conflict count are

$$L(P) = \textstyle\sum_{\{u,v\}\in E} p_u^\top p_v,$$
$$C(c) = \textstyle\sum_{\{u,v\}\in E} \mathbf{1}[c_u = c_v].$$

$p_u^\top p_v$ is the probability that u and v draw the same colour, so $L(P)$ is the expected conflict count when each vertex samples independently, and $L = C$ when every row is one-hot. PI-GNN decodes by argmax.

Local optimality and repair. Let $M_v(a)$ be the number of neighbours of v with colour a. Recolouring v can remove at most $\gamma_v = M_v(c_v) - \min_a M_v(a)$ conflicts; a colouring is *single-flip optimal* (SFO) if $\gamma_v = 0$ for all v. Our shared repair, *strict descent*, repeatedly applies a move with the largest positive γ_v and stops at an SFO colouring. We report raw conflicts C (how good a method's own answer is) and C_{desc} after descent (how good a *starting point* it is); in RQ2 these disagree.

Two facts frame the results: what an ideal endpoint looks like, and why training may not reach it.

Lemma 1. *If P^* is a strict local minimum of L over all row-stochastic matrices, every row is one-hot and every single-vertex recolouring strictly increases C.*

Sketch. With other rows fixed, $L = B + p_v^\top q_v$ where $q_v = \sum_{u\in N(v)} p_u$ is affine in p_v. Moving mass ϵ between two colours a, b in the support of p_v changes L by $\pm\epsilon(q_v(a) - q_v(b))$, so one of the two moves does not increase L, contradicting strictness. At a one-hot row, moving towards colour a changes L by $t\,[M_v(a) - M_v(c_v)]$, which must be positive. □

Training, however, moves logits, and for the Potts loss

$$\frac{\partial L}{\partial z_{v,a}} = p_v(a)\big[q_v(a) - p_v^\top q_v\big]. \qquad (1)$$

If p_v is nearly one-hot on colour c, then for $a \neq c$ the factor $p_v(a) \approx e^{-\Delta}$ is tiny, where Δ is the gap between the top two logits, and for $a = c$ the

bracket is $\approx q_v(c) - q_v(c) = 0$. All logit gradients of a *saturated* vertex vanish, even when another colour would remove many conflicts.

PI-GNN and benchmark. We reimplement PI-GNN in PyTorch: trainable Gaussian vertex embeddings (dimension 85), two mean-aggregation GraphSAGE layers (hidden width 30, ReLU, dropout 0.47), trained full-batch with AdamW (Loshchilov and Hutter, 2019). We use the 68 DIMACS graphs whose chromatic number is proven (Held et al., 2012; Gualandi and Malucelli, 2012), with 11 to 1,000 vertices from 14 families (queen, Mycielski, register allocation, Leighton, flat, DSJC/DSJR random, and others), each with 5 seeds for 10^5 steps, no early stopping, and dropout off at evaluation. Every model type is tuned separately by the same two-stage random search on 13 disjoint synthetic graphs, then frozen.

Comparison rule. The graph is the unit of analysis: we average the 5 seeds per graph, then compare paired graph means X_g, Y_g through $d_g = (X_g - Y_g)/\max(Y_g, 1)$. A graph is a practical tie if $|X_g - Y_g| < 1$ conflict or $|d_g| \leq 5\%$; otherwise a win or loss. We report wins/ties/losses (W/T/L), the median d_g, and a two-sided Wilcoxon signed-rank test (Wilcoxon, 1945). A directional claim requires a small p (Bonferroni-corrected over the primary comparisons of RQ1 and RQ2), a median beyond 5%, and more wins than losses; anything else is "not distinguishable", which is not a claim of equality. One rule serves all three RQs; it was fixed before the main RQ2 results were computed. The per-graph rule is primary because conflict counts span four orders of magnitude, so a mean is dominated by a few dense graphs.

4 Experiments

4.1 RQ1: What does training produce?

Figure 2 tracks the argmax output over training. The fraction of SFO runs rises from 0% to 42% (143/340), and the median share of conflicts that descent can still remove falls from 95% to 10%. Both curves flatten: from $5{\cdot}10^4$ to 10^5 steps the remaining gain does not move. Extending 13 still-improving graphs to $3{\cdot}10^5$ steps improves conflicts after descent on 12, yet SFO runs go from 0/65 to only 1/65. *Longer training finds better basins without reaching their bottoms.*

Usually only a few vertices are the problem (median 2.5 improvable vertices per run, under 1% of the graph), and those vertices are *confident*. In the median graph, 99.7% of vertices have $\max_a p_v(a) \geq 0.99$, and so do 100% of the improvable ones; their median top-two logit gap is about 150, so $p_v(a) \approx e^{-150}$ for the better colour. By Eq. (1) such a vertex gets almost no gradient towards the better colour: a plausible mechanism for frozen errors, though not a proven cause, since shared parameters can still move it indirectly.

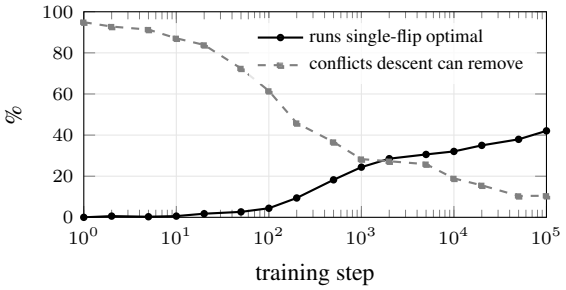


Figure 2: RQ1: PI-GNN's argmax output over training (68 graphs, 5 seeds). Solid: share of runs that are single-flip optimal. Dashed: median over graphs of the relative conflict reduction strict descent still achieves.

PI-GNN + descent vs.	W/T/L	median d
random colouring + descent	62/5/1	−38.7%
own epoch-1 output + descent	59/8/1	−32.5%
DSATUR + descent	16/28/24	0.0%
TabuCol, time-matched	14/13/41	–

Table 1: RQ1: quality of the trained endpoint at 10^5 steps, counted from PI-GNN's side (negative d favours PI-GNN). Mean conflicts: PI-GNN 114.9, TabuCol 70.5.

Is the endpoint at least good? Table 1 compares PI-GNN after descent with references given the same descent. Training clearly beats random and epoch-1 starts, but it is not distinguishable from the 1979 DSATUR heuristic, and a time-matched TabuCol finds fewer conflicts on 41 graphs. The queen family makes the point sharply: 85% of queen runs are already SFO, yet TabuCol wins on all 12 queen graphs. A local minimum can be shallow. Failure modes also differ by family: register-allocation runs are rarely SFO but only a few conflicts from it, while some dense DSJR runs stall near the uniform distribution, a stationary point of L.

4.2 RQ2: Is the network needed?

The graph enters the Potts loss, so a network is not needed to "use" it. The natural network-free competitor, arm B, optimises the logit matrix Z di-

rectly, starting from PI-GNN's exact initial output so that only the optimisation differs. Other arms each change one thing (Table 2), including Gaussian noise of scale σ added to Z during training (B-noise) and PI-GNN linearised at initialisation, $Z = Z_0 + J_0\delta$ with a frozen Jacobian (D), which tests whether the network acts as a fixed graph-dependent preconditioner. B-noise reuses B's tuning.

The verdict flips with repair. Free logits give *better raw* colourings than PI-GNN (32 wins, 13 losses, $p = 0.002$), but after descent PI-GNN is better (26 wins, 17 losses, median 6%, $p < 0.001$). Both readings hold at matched wall-clock time, where B gets about $2.9\times$ more steps and barely changes. The SFO column explains the flip: B ends single-flip optimal in all 340 runs, so descent has nothing to add, while PI-GNN stops short in 58% of runs but in basins whose bottoms are slightly deeper. B still has the lower *mean* after descent (108.6 vs. 114.9): PI-GNN's large relative wins come on low-conflict graphs, B's on high-conflict ones.

The advantage is noise, not architecture. Removing dropout from PI-GNN erases its edge: the result loses to PI-GNN on 21 graphs and is not distinguishable from B. Adding noise to free logits overturns it: at $\sigma = 2$ they beat PI-GNN after descent on 44 graphs, losing on 3, and beat plain B on 53 with no losses. A plausible reading is the one behind simulated annealing (Kirkpatrick et al., 1983): persistent noise prevents early commitment to the first basin. Indeed, at step 100, 69% of PI-GNN's vertices are already confident against 0.2% for B.

Noise, but of the right kind. Dropout and logit noise act in opposite directions on confidence. The arms with dropout (A and C) end with enormous logit gaps (204 and 139); without dropout the gap is 18, and with logit noise it shrinks to 8. We interpret this as the network inflating its weights to survive losing half its hidden units; with dropout off at evaluation every unit votes at full strength and outputs saturate, freezing the RQ1 errors (Eq. 1). Logit noise gives exploration without saturation, and the best arm is the *least* confident one.

Aggregation helps only inside the neural recipe; a frozen network fails. The node MLP C loses to PI-GNN after descent on 28 graphs (5 wins), so aggregation contributes *within* the neural recipe, although a simpler procedure does better without it. D is the worst arm (no wins against PI-GNN or B), so PI-GNN's help is not a fixed preconditioner from initialisation. B and B-rand are not distinguishable (14/42/12): the network's initial output gives no useful head start.

4.3 RQ3: Is it the soft output or the decoder?

A decoder that reads the graph can colour well even from a meaningless P, so we fix the decoder and vary its input. The decoder is derandomised conditional expectation (D-CE) (Raghavan, 1988; Karalias and Loukas, 2020): it fixes vertices one at a time in DSATUR order, giving v the colour a that minimises the number of fixed neighbours coloured a plus $\sum p_u(a)$ over unfixed neighbours u. It guarantees $C \leq L(P)$, and on exactly uniform P it reduces to DSATUR. A near-uniform input is thus a *graph-only* decoder with no learned information. We compare PI-GNN's trained P with it and with three corruptions: the untrained output, rows shuffled across vertices, and colours permuted independently within each row, which keeps every row's confidence but breaks agreement between neighbours.

Real information, but not more than the graph alone. Table 3 shows that trained P clearly beats every corruption, including the colour-scrambled one that preserves confidence exactly. The useful content is therefore *coordination* between neighbours, not confidence. Yet against the graph-only decoder the trained P is not distinguishable (median 0%, $p = 0.23$), consistent with RQ1's tie against DSATUR. Confidently wrong information is worse than none (near-uniform beats scrambled on 41 graphs), so beating a corrupted input is a weak test. The aggregate tie also hides a family split: trained P wins on all 12 queen graphs but loses to the graph-only decoder on DSJR, register-allocation and miles graphs.

Most of the decoder's gain is repair. D-CE improves raw argmax substantially, but once argmax also gets descent the gain is 2.8%, inside the tie band, though consistent and cheap. Best-of-32 sampling adds nothing (2/65/1) at $26\times$ the cost: saturated rows make sampling reproduce argmax.

The optimiser still decides. Decoding all nine RQ2 arms with D-CE narrows the spread between them (median within-graph range down by 10%)

Arm	change relative to A	vs. A, W/T/L (arm better / tie / A better) raw C	after descent	median d (desc.)	mean C_{desc}	SFO	logit gap
A	PI-GNN (reference)	–	–	–	114.9	42%	204
B	free logits from A's start	32/23/13	17/25/26	+6.0%	108.6	100%	18
B-rand	as B, random start	28/21/19	20/20/28	+10.8%	107.1	100%	19
A-no-drop	dropout removed	10/9/49	7/40/21	+8.7%	115.7	17%	18
B-noise-0.5	B + logit noise, σ=0.5	33/21/14	20/19/29	+5.8%	109.2	100%	19
B-noise-1	B + logit noise, σ=1	46/20/2	35/30/3	−15.9%	92.5	100%	13
B-noise-2	B + logit noise, σ=2	53/13/2	44/21/3	−27.5%	83.4	100%	8
C	node MLP: aggregation removed	20/28/20	5/35/28	+7.8%	120.4	41%	139
D	linearised at init, frozen J_0	14/12/42	0/10/58	+36.6%	130.4	42%	23

Table 2: RQ2 at 10^5 steps (68 graphs, 5 seeds). W/T/L and median $d = (\text{arm} - \text{A})/\max(\text{A}, 1)$ are graph-level; negative d favours the arm. SFO: share of runs already single-flip optimal. Logit gap: median top-two logit difference, a measure of saturation. B versus A is the primary comparison; the other rows are secondary.

D-CE + descent: first vs. second	W/T/L	median d
trained vs. colour-scrambled	48/20/0	−25.5%
trained vs. vertex-shuffled	56/12/0	−29.9%
trained vs. untrained	51/14/3	−22.2%
trained vs. near-uniform	27/28/13	0.0%
near-uniform vs. colour-scrambled	41/24/3	−17.5%
D-CE vs. argmax, raw	33/35/0	−11.4%
D-CE vs. argmax, after descent	22/46/0	−2.8%
B-noise-2 vs. near-uniform	36/27/5	−21.2%

Table 3: RQ3: probability-input and decoder controls on frozen 10^5-step outputs. "Trained" is PI-GNN's P unless stated.

but preserves the ranking: B-noise-2 still beats PI-GNN (35/29/4), D stays last, and B-noise-2 is the only arm whose P clearly beats the graph-only decoder. A better optimiser yields a better P, and that advantage survives decoding.

5 Discussion and Limitations

Returning to Figure 1, our answer to the research question is that **the optimiser decides the outcome, but the GNN is not what makes it good**. Differences between optimisers survive every decoder we tried, while PI-GNN's specific ingredients contribute little, and the strongest procedure we found has no network at all.

This sharpens the end-to-end critiques (Angelini and Ricci-Tersenghi, 2023; Boettcher, 2023). PI-GNN is not weak because it fails to learn: its P carries real coordinated structure (Table 3). It is weak because its optimisation saturates, leaving confident but improvable vertices (Eq. 1), and because what helps it, noise, is available more cheaply without a network.

For evaluating such methods we suggest: report raw *and* repaired quality, which can rank methods oppositely; use free logits on the same loss as the network-free baseline; test learned outputs against an uninformative input under the same decoder, not only corrupted ones; never treat confidence as correctness; and report per-family results.

Limitations. One architecture, one regime ($k = \chi - 1$) and 68 heterogeneous graphs limit generality. $\sigma = 2$ was the best of three values on the test graphs, not tuned on the synthetic set, and was not compared with TabuCol. Logit noise and dropout differ, so the noise account is plausible rather than isolated; saturation rests on evaluation-time diagnostics, not a training intervention; D is one linearisation, not every preconditioner. Time matching used shared mixed-GPU jobs, and our p-values are descriptive, with no confidence intervals.

6 Conclusion

Intervening on one ingredient at a time shows that PI-GNN's outputs become confident long before they become locally optimal, that the network's edge over plain logit optimisation is small and exceeded by simple noise, and that its learned probabilities are worth about as much as the graph itself given to a greedy decoder. Confidence, local optimality and quality are distinct, and conflating them overstates what the network contributes. A natural next step is to tune noisy logit optimisation and compare it with strong local search.

Acknowledgements

Implementation, experiment execution and drafting were carried out with substantial assistance from AI coding assistants. All numbers come from the accompanying code and results; the design, interpretation and final text are our own.

References

Maria Chiara Angelini and Federico Ricci-Tersenghi. 2023. Modern graph neural networks do worse than classical greedy algorithms in solving combinatorial optimization problems like maximum independent set. *Nature Machine Intelligence*, 5:29–31.

Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. 2021. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421.

Stefan Boettcher. 2023. Inability of a graph neural network heuristic to outperform greedy algorithms in solving combinatorial optimization problems. *Nature Machine Intelligence*, 5:24–25.

Daniel Brélaz. 1979. New methods to color the vertices of a graph. *Communications of the ACM*, 22(4):251–256.

Philippe Galinier and Jin-Kao Hao. 1999. Hybrid evolutionary algorithms for graph coloring. *Journal of Combinatorial Optimization*, 3(4):379–397.

Stefano Gualandi and Federico Malucelli. 2012. Exact solution of graph coloring problems via constraint programming and column generation. *INFORMS Journal on Computing*, 24(1):81–100.

William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

Stephan Held, William Cook, and Edward C. Sewell. 2012. Maximum-weight stable sets and safe lower bounds for graph coloring. *Mathematical Programming Computation*, 4(4):363–381.

Alain Hertz and Dominique de Werra. 1987. Using tabu search techniques for graph coloring. *Computing*, 39(4):345–351.

Arthur Jacot, Franck Gabriel, and Clément Hongler. 2018. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31.

David S. Johnson and Michael A. Trick. 1996. Cliques, coloring, and satisfiability: Second DIMACS implementation challenge. *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, 26.

Nikolaos Karalias and Andreas Loukas. 2020. Erdős goes neural: an unsupervised learning framework for combinatorial optimization on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6659–6672.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. 1983. Optimization by simulated annealing. *Science*, 220(4598):671–680.

Henrique Lemos, Marcelo Prates, Pedro Avelar, and Luís Lamb. 2019. Graph colouring meets deep learning: Effective graph neural network models for combinatorial problems. In *IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, pages 879–885.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.

Prabhakar Raghavan. 1988. Probabilistic construction of deterministic algorithms: Approximating packing integer programs. *Journal of Computer and System Sciences*, 37(2):130–143.

Martin J. A. Schuetz, J. Kyle Brubaker, and Helmut G. Katzgraber. 2022a. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence*, 4:367–377.

Martin J. A. Schuetz, J. Kyle Brubaker, Zhihuai Zhu, and Helmut G. Katzgraber. 2022b. Graph coloring with physics-inspired graph neural networks. *Physical Review Research*, 4:043131.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.

Knut Vanderbush and Melanie Weber. 2026. Neural algorithmic reasoning for approximate k-coloring with recursive warm starts. *arXiv preprint arXiv:2601.05137*.

Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83.

Fa-Yueh Wu. 1982. The Potts model. *Reviews of Modern Physics*, 54(1):235–268.